

WEBSITE TECHNICAL SPECIFICATION

SYSTEM ARCHITECTURE & MANUAL

Project Name:	WealthFlow / VelocityWealth Platform
Core Stack:	React v19, TypeScript, Express v4, Tailwind C
Environment:	Google Cloud Run (Docker-Managed, Port 300
Version Status:	v1.4.0 (Enterprise Deployment Ready)
Target Date:	August 25, 2026

The WealthFlow platform is engineered as a highly responsive, modern, full-stack single-page application (SPA). It is built with React 19 and Vite on the client side, connected directly to a high-performance Express.js server on the backend. It integrates seamlessly with Google Cloud Run containers and utilizes Firebase Firestore and Auth as its secure persistent data cloud layer.

1.1 Operational Port Mapping

- Port Binding: The Node server binds strictly to port 3000.
- Ingress Layer: Reverse proxy layers direct all public traffic on HTTPS to port 3000.
- Asset Pipeline: Client bundles are served statically, while APIs route to the proxy controllers.

2.1 Identity & Encryption Perimeter

Security is integrated directly into both the transport layer and the transaction endpoints. All data transfer is wrapped in SSL/TLS. Firebase Auth tokens are passed dynamically from the client in authorization headers and verified on the server side using the secure Admin SDK.

2.2 Firestore Database Collection Schemas

- Collection [users]: Holds user metadata, system roles, verification tokens.
- Collection [holdings]: Logs fund assignments, NAV valuations, portfolios.
- Collection [transactions]: Immutable record logs for orders, payouts, and pending states.
- Collection [sips]: Client automated periodic investment schedules.

3.1 Backend Proxy & Integration Endpoints

- GET /api/market-indices: Calls Yahoo Finance API internally.
- GET /api/stocks/search: Filters and parses live equities and NAVs.
- POST /api/payments/create-order: Interfaces with Razorpay Checkout sandbox.
- GET /api/mutual-funds/details: Extracts direct performance data from Indian Mutual Fund API.

3.2 DevOps Build & Run Specification

The container uses a custom layered Dockerfile build pipeline. The server is dynamically compiled with esbuild to target standard CommonJS (dist/server.cjs) output, resolving all modular relative paths at compile time. This bypasses runtime Node relative loading constraints and guarantees extremely fast startup times under 100ms.